DAY - 03
Activity 1
Waterfall and its Workflow

Waterfall Model:

1. It is a linear and sequential software development model.
2. Each phase must be completed fully before starting the next phase.
3. No going back to previous phases once completed.
4. Phases follow a fixed order:
   a. Requirements
   b. Design
   c. Coding
   d. Testing
   e. Deployment
   f. Maintenance
5. Best suited for projects with clear and stable requirements.
6. Heavily documentation-based process.

Advantages:

1. **Well-organized workflow and strong documentation**: each stage is clearly defined, making it easier for new team members to understand the project and maintain proper records.
2. **Reliable scheduling and budgeting**: because requirements are finalized at the beginning, timelines and costs can be estimated more accurately.
3. **Simple to oversee**: the structured, phase-by-phase approach makes project management straightforward, with clear checkpoints at every step.

Disadvantages:

1. **Defects are discovered late**: because testing happens only after development, issues found near the end become significantly more costly and time-consuming to correct.
2. **Poor adaptability to requirement changes**: if business priorities shift, the entire project plan may need major revisions or even a full restart.

ACTIVITY 2
AGILE AND ITS TECHNIQUES

Agile is a way of working where teams:

- Work in small steps
- Check their progress often
- Fix problems quickly
- Change direction easily when needed
- Talk to each other regularly

Agile helps teams deliver work **faster, better, and with fewer mistakes**.

Agile Manifesto

The **Agile Manifesto (created in 2001)** outlines 4 core values.

Four Core Values

1. **People and communication** are more important than rigid tools and processes.
2. **Working software** matters more than extensive documentation.
3. **Customer involvement** is valued more than contract negotiations.
4. **Adapting to change** is more important than strictly following a fixed plan.

Agile Methodologies

1. **Scrum**

   Scrum breaks work into short cycles called **sprints** (usually 2 weeks).

   1. Product log: A list of all desired features, prioritized by value.
   2. Sprint Planning: The team picks a small batch of features to complete in the next 2 weeks.
   3. Daily Stand-up: A quick 15-minute sync to discuss progress and blockers.
   4. The Sprint: Developers and designers work to create a functional

ACTIVITY 3
CORE SCRUM MANAGEMENT TOOLS

Agile Scrum framework relies on continuous collaboration, iterative development, and rapid feedback.

Core Scrum Management Tools

1. **Jira & Azure DevOps (ADO):**
   - ➔ Issue tracking, backlog grooming, CI/CD pipeline integration, and velocity reporting.
   - ➔ These tools are like a command centre where all projects' tasks are stored, assigned, and tracked from start to finish.
2. **Confluence & Notion:**
   - ➔ Asynchronous collaboration, maintaining the definition of Done (DoD), and API documentation.
   - ➔ The project's digital library where all rules, requirements, and reference materials are securely stored.
3. **Miro & Figma:**
   - ➔ UI/UX wireframing, architecture mapping, and digital whiteboarding for Retrospectives.
   - ➔ It's mainly used for brainstorming sessions and designing what the application will look like.

AI-Specific Scrum Enhancements

1. **Atlassian Jira Ai:**
   Backlog Refinement: Auto drafts user stories, suggests acceptance criteria, and summarizes long comment threads so developers do not have to read through weeks of back-and-forth
2. **Zoho sprints:**

   Artifact Generation: Automatically generates checklists for the "Definition of Done" and acceptance Criteria based on the title and description of a work item.
3. **ScrumGenius:**

ACTIVITY 4

BLACK, WHITE, & GRAY BOX

White Box Testing

White Box Testing is a technique where the tester has **complete visibility** into the application's internal code, architecture, and logic.
Also known as **Clear Box** or **Glass Box Testing**, this method focuses on verifying the internal processes to ensure they function exactly as intended.
Test cases are designed based on internal code paths, conditions, branches, loops, and program structure—not just expected outputs.

Practices Used

- **Statement Coverage** – Ensures every line of code is executed at least once.
- **Branch Coverage** – Tests every possible decision path (e.g., all true/false outcomes).
- **Path Coverage** – Validates all possible execution flows through the code.
- **Static Analysis & Code Review** – Checks the code without running it to identify syntax issues, security flaws, and coding standard violations.

Who Performs It?

Primarily executed by **developers** or **technical QEs** who have strong programming skills and direct access to source code.

Tools Used

- **JUnit / TestNG** – For Java unit testing at method/class level.
- **NUnit / xUnit** – Unit testing frameworks for .NET applications.

Black Box Testing

Black Box Testing is a method where the tester evaluates the software **without any knowledge of the internal implementation**.
The application is treated as a "black box," where only inputs, outputs, and user-facing behavior matter.
Test cases come directly from **requirements, user stories, or functional specs**.

ACTIVITY 5

System Assumptions for Testing

- Minimum Withdrawal: ₹100.
- Maximum Withdrawal: ₹10,000.
- Dispense Rules: Amounts must be in multiples of ₹100.

Equivalence Partition

Grouping infinite input possibilities into valid and invalid buckets and testing just one item from each bucket to prove the whole group works.

It guarantees high test coverage while drastically cutting down the number of tests that testers have to execute.

Test ID	Partition	Input Value	Expected Result
TC01	Valid Multiple of 100	₹500	Success. Cash Dispensed
TC02	Below Minimum	₹10	Error: Minimum withdrawal is 100
TC03	Above Maximum	₹20,00	Error: Exceeds maximum limit.
TC04	Not a multiple of 100	₹55	Error: Not a multiple of 100
TC05	Non-numeric	"ABC"	Error: Non-numeric values are invalid

Boundary Value Analysis (BVA)

Testing the extreme limits of an input range (the exact min, max) because software is most likely to fail at the edges.